דף סיכום בחינה

מזהה סטודנט:

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר שאלה	ניקוד מירבי	ציון
1	23.00	18.00
2	16.00	15.00
3	16.00	16.00
4	5.00	5.00
5	5.00	5.00
6	5.00	4.00
7	5.00	5.00
8	5.00	4.00
9	5.00	5.00
10	5.00	0.00
11	5.00	5.00
12	5.00	5.00

ציון בחינה סופי : 87.00

הבחינה הבדוקה בעמודים הבאים

[redacted]

מספר התלמיד הנבחן
רשום את כל תשע הספרות

[redacted]

האוניברסיטה הפתוחה

ו' באדר תשפ"ג

מס' שאלון - 516

27 בפברואר 2023

מס' מועד 91

סמסטר 2023א

20594 / 4

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **10** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות ולהקיף בעיגול את אות התשובה שבחרתם.

את התשובות לכל השאלות יש לכתוב בשאלון הבחינה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

mutex1- מגן על read count
mutex2- מגן על read count
w- כותב אחד בכל פעם

mutex3- מגבל את הקוראים
r - מונע את הקוראים

חלק א - 55 נקודות

שאלה 1 (23 נקודות)

נתון הפתרון הבא של בעיית קוראים-כותבים למספר תהליכונים THREADS הרצים במקביל.
כל הסימפורים SEMAPHORES הם בינאריים ולכן אצל חלקם מופיע שם MUTEX .

```
int read_count=0, write_count = 0;
semaphore mutex1=1, mutex2=1, mutex3=1, w=1, r =1;
```

```
writer() {
        wait(mutex2);
        write_count++;
        if( write_count == 1 )  wait(r);
        signal(mutex2);
        wait(w);
        //do writing כתיבה מתבצעת
        signal(w);
        wait(mutex2);
        write_count--;
        if (write_count == 0) signal(r);
        signal(mutex2);
}
```

```
reader() {
        wait(mutex3);
        wait(r);
        wait(mutex1);
        read_count ++;
        if(read_count == 1) wait(w);
        signal(mutex1);
        signal(r);
        signal(mutex3);
        //do reading קריאה מתבצעת
        wait(mutex1);
        read_count--;
        if (read_count == 0) signal(w);
        signal(mutex1);
}
```

18
(1)

א. (9 נק') תארו כיצד פועל האלגוריתם הזה לפתרון בעיית הקוראים כותבים? הוכיחו שהוא אכן פותר את הבעיה או הפריכו ע"י דוגמא נגדית

*הנחתי ש- wait + signal שקולות לפעולות up + down.

האלגוריתם עובד באופן הבא: עבור הכותבים, כל עוד קיים קורא בקטע הקריטי. הכותבים ימתינו על הסמפורים r, mutex2, בכל פעם שכותב ממתין לבצע כתיבה, הוא עושה זאת ע"י הגדלת write_count והמתנה על הסמפור r. ברגע שהקטע הקריטי נקי מקוראים, והאחרון עשה signal(w), אז הכותב יכול לבצע את עבודת הכתיבה. רק כותב אחד יכול לבצע כתיבה בכל פעם ע"י המתנה על הסמפור w. עבור הקוראים, הם ממתינים כל עוד יש כותב בקטע הקריטי ע"י המתנה על הסמפור w. יכולים להיות מס' קוראים בו זמנית בקטע הקריטי. הקוראים יכולים לחסום כותבים ע"י כך שהקורא הראשון יעלה את המונה read_count ויבצע wait(w). האלגוריתם הנ"ל פותר את בעיית קוראים כותבים כיוון שמובטחת מניעה הדדית בין הקוראים והכותבים. קורא לא ייכנס לקטע הקריטי כל עוד יש כותבים, זאת בזכות הסמפור w והעובדה ש-read_count מוגן ע"י הסמפור mutex1, וכותב לא יוכל להיכנס לקטע הקריטי כל עוד יש קורא עקב המתנה על הסמפור r והעובדה ש-write_count מוגן ע"י mutex2.

2

כשיש כותב שתפס R, קורא לא יוכל להצטרף למרות שיש כבר קוראים

שאלון 516
91.66.8 M2

טיוטה

תנאים לקיפאון:

1) מניעה הדדית
2) החלקה והמתנה
3) אין חילוף כפוי

הכרחי

4) מעגל - מספיק

מניעה הדדית - משאב מוחזק ע"י תהליך אחד בלבד או משוחרר.

החלקה והמתנה - התהליך יתפוס את המשאב לא משחרר

ב. (8 נק') האם האלגוריתם יכול לגרום להרעבת חלק מתהליכונים THREADS ? הוכיחו שלא או תביאו דוגמא שכן

כן. האלגוריתם יכול לגרום להרעבה. הרעבה זה מצב שבו תהליך ממתין
למשאב כלשהו התפוס ע"י תהליך אחר. אם למשל יש המון תהליכי reader שמגיעים
הפוך
בזה אחר זה. ובשלב כלשהו יגיע תהליך writer, הוא ימתין על הסמפור
r כל עוד אחרון תהליכי ה-reader לא שחרר את r. תהליך writer
לא יוכל להתקדם, כיוון שהוא ממתין למשאב שישתחרר,
וזה מצב של הרעבה.

כשיש כותבים שמחכים, קוראים לא יכולים להצטרף בגלל R

✓

ג. (6 נק') האם האלגוריתם יכול לגרום לקיפאון DEADLOCK ? הוכיחו שלא או תביאו דוגמא שכן.

האלגוריתם לא יכול לגרום לקיפאון, מאחר ולא יכול להתקיים התנאי
של החזקה והמתנה (תנאי הכרחי לקיפאון). המשאבים היחידים הם הסמפורים
w+r. מאחר ויש מניעה הדדית בתוך כל תהליך על הסמפורים w, r
מובטח שברגע שקורא ממתין על r או כותב ממתין על w, התהליך
שתפס את הסמפור r או w ישחרר אותו. וכך התהליך הבא בתור
יוכל לבצע את עבודתו. כתוצאה מכך לא נגרם מעגל המתנה
ולא יכול להתרחש deadlock.

✓ **שאלה 2 (16 נקודות)**

נתון זיכרון פיזי (RAM) בגודל של 16MB (קטן בשביל פשטות החישובים).

המהנדסים החליטו לבנות את מערכת הזיכרון עם מסגרות (frames) בגדלים שונים.

הדפים/המסגרות יהיו בגדלים הבאים: 1k , 2k , 4k ו 8k . הרבע הראשון של הזיכרון יחולק לגודל של 1k, הרבע השני יחולק לגודל של 2k, הרבע השלישי יחולק לגודל של 4k והרבע הרביעי יחולק לגודל של 8k.

$16MB = 16 \times 2^{20} B = 2^{24}$

א. (6 נק')

בהינתן הכתובת הבאה(בייצוג בינארי):

6307329

0	1	1	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	1

מה גודל, מספר המסגרת וה OFFSET שבכתובת הנ"ל?

$2KB = 2\times 2^{10}B = 2^{11}B$

הומר מבינארי לבסיס 10 במחשבון

הסבר וחישוב:

הכתובת בבסיס 10 היא 6307329 והיא נמצאת בפרק 6MB מתחילת הזיכרון. לכן היא נמצאת ברבע השני, כלומר גודל הדף הוא 2KB. לכן צריך 11 סיביות $(2KB=2^{11}B)$ לייצוג מס' המסגרת/דף. ו- 13 סיביות עבור ההיסט.

לכן מס' המסגרת הוא $(01100000001)_2 = (769)_{10}$

וההיסט הוא $(1111000000001)_2 = (7681)_{10}$

15
(2)

ב.(5 נק')

נתון כי לכל גודל יש לפחות 10 מסגרות פנויות בזיכרון. המשתמש הפעיל תכנית בגודל של 6.5K,
כיצד היית מקצה את הזיכרון לתכנית זו?

הסבר וחישוב:

השיקולים שלי הם: צמצום ריסוק פנימי, וחלוקה של כמה שפחות דפים. לכן, הייתי מקצה לתכנית דף אחד בגודל 4KB, דף נוסף בגודל 2KB ודף אחרון בגודל 1KB. כך כמות הדפים והריסוק הפנימי מינימליים עבור התכנית הזו. סה"כ הוקצה 7KB ובוזבז 0.5KB (ריסוק פנימי של 0.5KB).

ג. (5 נק')

מה הוא היתרון של השיטה הזאות(חלוקת הזיכרון למסגרות בגודל שונה)?

השיטה הזו מצמצמת ריסוק פנימי מכיוון שניתן לצמצם את הזיכרון המבוזבז ע"י התאמה של הדף האחרון שמוקצה לזיכרון שנשאר.
כמו כן ניתן לשמור טבלת דפים קטנה יותר בזיכרון מכיוון שעם הגדלת גודל דף, כמות הדפים קטנה יותר.

למה זה חשוב?

המשך הבחינה בעמוד הבא

שאלה 3 (16 נקודות)

נתונה מערכת קבצים מבוססת NODES– I עם 12 מצביעים ישירים, והשאר המצביעים הרגילים.

במערכת זו מטמון מערכת קבצים מבוטל.

גודל הבלוק הוא 1 KB , אורך הכתובת 4 BYTE .

$\frac{1KB}{4B} = \frac{2^{10}}{2^2} = 2^8 = 256$

כמות מצביעים בבלוק:

א. (7 נק')

לכן בלוק Single indirect יכול להצביע ל- 256KB של מידע.

התוכנית מגרילה בכל פעם באופן רנדומאלי אה המיקום(offset) בקובץ נתון וקוראת 100 בתים של נתונים ממיקום זה בקובץ.

הניחו כי לכל קובץ ממנו קוראת התוכנית מוקצה מקום רציף בדיסק.

לאחר הרצת התכנית מספר רב של פעמים על קבצים שונים שמו לב לתופעה הבאה:

משך הזמן הנדרש להרצת התכנית שונה עבור קבצים בגדלים שונים.

עבור קבצים קטנים(100KB לכל היותר) משך ההרצה קצר בצורה משמעותית מקבצים גדולים (יותר מ 1GB) . כיצד ניתן להסביר תופעה זו בהסתמך על מאפייני המערכת? הסבירו בקצרה אך במדויק.

תשובה והסבר עבור קבצים קטנים (עד 100KB) מתבצעת גישה לבלוקים שמוצבעים לכל היותר ברמה של Single indirect. לעומת זאת, בקבצים גדולים, מתבצעת גישה לבלוקים שהם Triple indirect. הבדל זה גורם לכך שחיפוש וגישה לבלוקים של הקבצים גדולים יארך יותר זמן, כי צריך לעבור דרך המון מצביעים עד שמגיעים לבלוק עצמו.

ב. (4 נק')

האם התופעה הנזכרת לעיל בהכרח תהיה קיימת באותה מידה במערכת שמחזיקה את הקבצים לפי הגודל שלהם בדרך הבאה:

הקבצים הגדולים בתחילת הדיסק הקשיח והקטנים בסופו. אין צורך להתייחס לשאלה איך היא עושה את זה.

תשובה והסבר התופעה לא בהכרח תתקיים במקרה הזה. כאן, יש את עניין הזזת הראשים בדיסק (seek time), שיכול לקחת המון זמן. אם למשל ראש הדיסק נמצא בהתחלה, ומתבצעת גישה לקובץ שנמצא בסוף הדיסק, הדבר ייקח הרבה יותר זמן מאשר מעבר על המצביעים ברמות השונות של ה-inode-ים. כי תזוזת ראש הדיסק תיקח מס' מילי שניות. לכן במצב זה גישה לקבצים קטנים תיקח יותר זמן.

1024-12=1012
1012-256=756

$\left\lceil \frac{756}{256} \right\rceil = 3$

ג. (5 נק')

16
(3)

תוכנית חיפוש של תבניות(רצפים של תווים) בקבצים במערכת הנ"ל מביאה את הקבצים לזיכרון. מהו מספר הבלוקים המקסימאלי שצריך לקרוא מתוך מערכת הקבצים הנ"ל לזיכרון על מנת לבצע חיפוש בקובץ שגודלו 1024 KB?

חישוב והסבר על מנת לאחסן בדיסק קובץ בגודל 1024KB,
(3) רק: (1) 12 בלוקים ישירים (2) בלוק Single indirect ועוד
256 מצביעים בבלוק זה (3) בלוק double indirect, 3
בלוקים של single indirect ועוד 756 מצביעים ישירים.
לכן קובץ זה מאוחסן על פני 12+(1+256)+(1+3+756) = 1029
בלוקים. נוסיף לזה את ה- i-node עצמו, ונקבל
שהמערכת תצטרך לחפש על פני 1030 בלוקים.
(ולקרוא אותם כמובן).

חלק ב - 15 נקודות, 5 נקודות לכל שאלה

שאלה 4

5
(4)

למה יש צורך בהקצאת מספר לקריאת מערכת SYSTEM CALL ? האם ניתן לממש את קריאת מערכת בלי שיהיה לה מספר?

תשובה והסבר המספר של קריאת מערכת הוא המזהה הייחודי שלה. בעת ביצוע
קריאת מערכת, מתבצע Trap ל- Kernel, וחלק מהמידע המועבר ב- Trap
הוא מספר קריאת המערכת - כך ה- Kernel "יודע" איזה שירות לספק לתהליך
שביצע את ה- Trap. אפשר לממש קריאת מערכת בלי שיהיה לה מספר, אך
זה מאוד לא פרקטי ויקר, למשל ע"י הקצאה של רגיסטרים ב- CPU לכל
סוג של קריאת מערכת. עדיף שיהיה לה מספר.

שאלה 5

אחד מעקרונות הקונטיינרים שתהליך משתמש לא יכול לגלות שהוא רץ בקונטיינר. איך ייתכן שתהליך שרץ במרחב המשתמש (למשל פקודת PS שנוספה בממ"ן 12) יכול לגלות את ה PID "האמיתי"(לא בתוך קונטיינר) שלו ושל אחרים? מי "אחראי" על ה"כשל"?

5
(5)

תשובה והסבר: האחראית לכשל היא קריאת המערכת שפקודת PS עוטפת.
בעת ביצוע פקודת PS מתבצעת קריאת מערכת הניגשת
למבני הנתונים הפנימיים של Kernel ושולפת את המידע על
התהליכים. מכיוון שקריאות המערכת הללו לא מודעות לכך שהתהליך
נמצא בתוך הקונטיינר. היא מספקת לו את ה- PID האמיתי
שלו ושל שאר התהליכים במערכת.

✓

שאלה 6

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

unshare(MOUNT_NS) נקרא אחרי ביצוע
fork ← נוצר mount ns חדש

```
int pid = fork();
if (pid == 0) {
  mkdir ("dirS");
  unshare (MOUNT_NS);
  mount ("internal_fs_a","dirS",0); //mount a preformatted internal_fs_a
  int fd = open ("dirS/file.txt",O_CREATE|O_RDWR);
  write (fd ,"123456789\n",10);
  close (fd);
 }
if (pid > 0) sleep (10000); // wait for child to finish
sleep (10000); // both processes will sleep
if( pid == 0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to open\n");
   else printf (stderr, "son succeeded\n");
 }
If (pid>0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to open\n");
   else printf (stderr, "father succeeded\n");
 }
```

תשובה והסבר:

unshare(MountNS) נקרא לאחר ביצוע fork() ולכן נוצר mount ns חדש אצל תהליך הבן, אבל התיקיה dirS נוצרה לפני ה-unshare ולכן היא לא ב- mount ns החדש. לכן הקובץ זמין גם לאב.

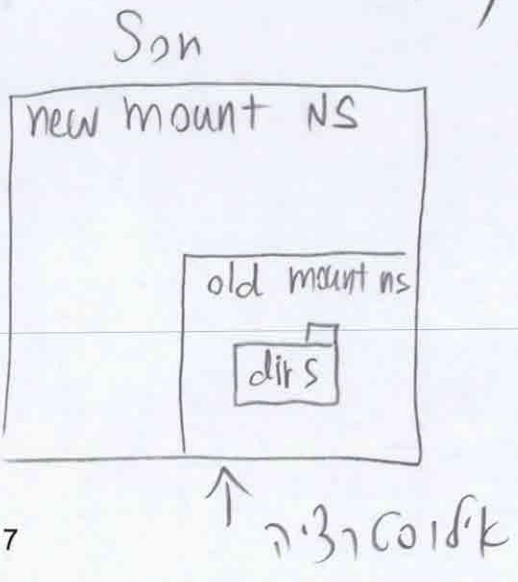
לכן יודפס:

son succeeded

~~father succeeded~~

MOUNT_NS נפרד בגלל MOUNT

4
(6)



המשך הבחינה בעמוד הבא

חלק ג (30 נקודות)

אם הציון 0 הוא ליד תשובה נכונה

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולהקיף בעיגול את אות התשובה שבחרתם.

שאלה 7

איזה מהטיעונים הבאים נכון לגבי תזמון דיסק (Disk Scheduling) בשיטת חיפוש קצר קודם SSF ?

5
(7)

א. מנצל את שטח הדיסק בצורה אחידה והוגנת.

ב. מבטיח העדר הרעבה של פעולות קלט/פלט(I/O) בכל הקבצים הרלוונטיים.

[✓ ג.] בודק את בקשות קלט/פלט הממתינות לדיסק כדי לקבוע את הסדר היעיל ביותר לשרתם.

ד. מחלק בצורה שווה ואחידה זמן לביצוע פעולות קלט/פלט עבור כל הקבצים הרלוונטיים

שאלה 8

מה התיאור הנכון של השתלשלות הפעולות מאז ביצוע SYSTEM CALL כתיבה לקובץ ע"י תהליך המשתמש במערכת LINUX אורחת GUEST ווירטואלית שרצה על תוכנת VMWARE שנשענת על טכנולוגיית וירטואליזציה VT שמופעלת על מערכת WINDOWS מארחת HOST. שימו לב איזה KERNEL "מנסה" לבצע את הפקודה ומה קורה בפועל.

[✓ א.] ה SYSTEM CALL מתבצע ע"י KERNEL של מערכת אורחת GUEST עד הסוף(כולל כתיבה בפועל)

אם למערכת אורחת יש מקום בדיסק הווירטואלי שלה .

א. ב. ה SYSTEM CALL מתבצע ישירות ע"י KERNEL של מערכת מארחת HOST בגלל שבזמן התקנת מערכת אורחת (LINUX) VMWARE החליפה את כל קריאות מערכת שלה לקריאות מקבילות של WINDOWS .

ב. ניסיון כתיבה לדיסק יגרום להפעלת מודול MODULE ב KERNEL של מערכת המארחת HOST שנוסף שם ע"י VMWARE בזמן ההתקנה או היה שם קודם, והוא מעביר אותה ל KERNEL המארח HOST.

ג. ניסיון כתיבה לדיסק יגרום להפעלת מודול MODULE ב KERNEL של מערכת האורחת GUEST שנוסף שם ע"י VMWARE בזמן ההתקנה או היה שם קודם, והוא מבצע את הכתיבה בפועל.

הסבר: כתיבה לדיסק היא הוראה רגישה (PI) ולכן קריאת המערכת תתבצע בצורה שונה ב-user mode ו-kernel mode. מכיוון שהמעבד תומך ב-VT, ה-Guest יוכל לבצע את קריאות המערכת ישירות (ההוראה ממופת באמצעות מפת סיביות חומרתית)

4
(8)

המשך הבחינה בעמוד הבא

שאלה 9

מהי הסיבה העיקרית לשימוש ב-DMA?

א. למעבד דרוש פחות זמן לביצוע כל פקודה בודדת.
[✓ ב.] שיפור ביצועי המערכת ע"י הגדלת המקביליות.
ג. הקטנת העומס על הזיכרון הראשי.
ד. מאפשר להתקן DEVICE, למשל דיסק, להשתמש ב RAM כמטמון.

5
(9)

שאלה 10

הבדל אחד בין תהליך PROCESS לבין תהליכונים של גרעין kernel thread הוא:

א. עם ריבוי תהליכים ניתן לנצל ריבוי מעבדים, אבל ע"י kernel threads לא.
ב. kernel threads מונעים את הבעיה של חסימה כאשר אחד מבצע פעולת I/O , אבל הבעיה קיימת כשמשתמשים בריבוי תהליכים.
ג. תהליכים תמיד זקוקים לתיווך של מערכת ההפעלה כדי לתקשר ביניהם ו kernel threads לא תמיד.
[✓ ד.] kernel threads מריצים רק קוד של מערכת ההפעלה, ואילו תהליכים יכולים להריץ גם קוד משתמש.

0
(10)

שאלה 11

באיזה מהמצבים הבאים בהם קיים מעגל בגרף הקצאות ודרישות **המשאבים בפועל(בנקודת זמן מסוימת), ניתן לומר בוודאות** שיש קיפאון deadlock ?

✗ א. כשיש תהליך במערכת שאינו חלק מהמעגל.
✗ ב. כשיש משאב במערכת שאינו חלק מהמעגל.
[✓ ג.] כשלכל אחד מהמשאבים במערכת יש רק מופע אחד.
✗ ד. כשלכל אחד מהמשאבים במערכת יש יותר ממופע אחד.

5
(11)

המשך הבחינה בעמוד הבא

שאלה 12

ניתן ליצור איזור של זיכרון משותף SHARED MEMORY בין שני תהליכים ע"י יצירת שתי טבלאות דפים שבכל אחת יש מצביעים לאזור המשותף של שני התהליכים . במצב זה:

X א. האזור המשותף חייב להיות ממופה **לאותה כתובת וירטואלית** במרחבי הכתובות של שני התהליכים. לא, ה- Kernel מחליט לאיזה מסגרת כל דף ממופה

X ב. אם אחד התהליכים פונה לכתובת משותפת ויש page fault , אזי שני התהליכים יחסמו **בוודאות**. לא, רק התהליך שגרם לפסיקת דף

[✓ ג.] אם יש במערכת רק מעבד אחד, העברת נתון מתהליך אחד לשני דרך האזור המשותף(עד שהתהליך השני יקרא את הנתון) **לפעמים** תיקח יותר זמן מאשר העברת הנתון דרך משתנה משותף בין תהליכונים THREADS שונים של אותו תהליך. → כן, אם יש פסיקת דף

5
(12)

X ד. אם יש במערכת רק מעבד אחד, העברת נתון מתהליך אחד לשני דרך האזור המשותף(עד שהתהליך השני יקרא את הנתון) **בוודאי** תיקח יותר זמן מאשר העברת הנתון דרך משתנה משותף בין תהליכונים THREADS שונים של אותו תהליך.

↑ אם הנתון נמצא ב- cache אז לא מהכרח

בהצלחה!